SASTRA DEEMED TO BE UNIVERSITY
Kumbakonam

Department of Computer Science and Engineering

SmartDock

A Multi-Tenant Dock Appointment Scheduling and Yard Management System for Warehouses and Distribution Centres

A project report submitted in partial fulfilment of the requirements for
Supply Chain Management – Final Project

B.Tech

Submitted by

Ramaguru S
Register No.: 227003181

Under the guidance of

Dr. ALAMELU R

Academic Year 2026–2027
November 2026

Abstract

The loading dock is the physical interface between a warehouse and the transport network, and it is one of the most frequent bottlenecks in a supply chain. When carriers arrive without appointments, trucks queue in the yard, drivers accumulate unpaid waiting (detention) time, dock labour is idle in quiet hours and overloaded at peaks, and the warehouse has little data with which to manage carrier performance.

This project presents **SmartDock**, a cloud-based, multi-tenant *dock appointment scheduling and yard management system*. Warehouse and distribution-centre (DC) managers configure their dock doors (inbound/outbound, supported trailer types, hazmat certification and capacity), operating hours, slot length, turnover buffers, holidays and maintenance windows. Carriers request access to a warehouse, then book a compatible dock slot through a guided wizard, optionally as a weekly recurring appointment; when a slot is full they can join a first-in-first-out waitlist that is served automatically when capacity is released. On the day, the yard is executed through gate check-in, dock start and completion events, from which the system derives on-time performance, no-shows, turnaround time and a carrier reliability score. An analytics module reports dock utilization, demand heatmaps, busiest hours and carrier scorecards, with CSV/PDF export and a printable gate sheet. Every action is recorded in an audit trail and communicated through in-app, e-mail and SMS notifications.

The system is built on the MERN stack (MongoDB, Express, React, Node.js) in TypeScript and is deployed on Render with MongoDB Atlas. Particular attention was paid to correctness: the booking engine guarantees that the capacity of a dock slot is never exceeded, even when requests arrive simultaneously on several server instances, using a MongoDB transaction with a dock-level write lock. An automated end-to-end test suite of 58 checks validates security, concurrency, scheduling rules and the business workflows; a controlled experiment showed that the naive approach admitted 15 trucks into a single-truck slot under 40 concurrent requests, whereas SmartDock admitted exactly one. A discrete-event simulation of a six-door DC handling 60 trucks per day indicates that appointment scheduling reduces the average yard waiting time from 19.4 minutes to about 1 minute and the share of trucks waiting more than 30 minutes from 27.7% to 0.3%.

Keywords: supply chain management, dock appointment scheduling, yard management, truck appointment system, warehouse operations, carrier performance, MERN stack, cloud deployment.

Source code: `https://github.com/RamaguruS/Dock` **Live system:** `https://smartdock.onrender.com`

Contents

Abstract **i**

1 Introduction **1**
1.1 Role of Information Technology in Modern Supply Chains 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations 3

2 Background and Literature Survey **4**
2.1 Background: Docks, Yards and Appointments 4
2.2 Literature Survey 4
2.3 Existing Systems 5
2.4 Gap in Existing Systems 6
2.5 Proposed Solution 6

3 System Design **7**
3.1 System Architecture 7
3.2 Data Flow Diagrams 8
3.3 Use Case Diagram 9
3.4 Entity–Relationship Diagram 9
3.5 Appointment Lifecycle 9
3.6 Technology Stack 11

4 Modules Description **12**
4.1 Identity and Access Management 12
4.2 Site and Dock Configuration 13
4.3 Scheduling Engine 13
4.4 Appointment Booking 13
4.5 Waitlist 13
4.6 Yard Execution 14
4.7 Carrier Performance 14
4.8 Analytics and Reporting 14
4.9 Notifications 15
4.10 Audit Trail 15

5 Implementation Details **16**
5.1 Project Structure 16
5.2 Time-Zone-Correct Slot Generation 16
5.3 Load–Dock Compatibility 17

5.4 Atomic Capacity Reservation 17
5.5 Recurring Bookings and Rescheduling 18
5.6 Waitlist Auto-Assignment 18
5.7 Arrival Classification and Background Jobs 18
5.8 Security Measures 19
5.9 Deployment 19

6 Screenshots and Live Demo **20**

7 Testing and Validation **25**
7.1 Testing Strategy 25
7.2 Test Cases 25
7.3 Concurrency Experiment 27
7.4 Performance 27
7.5 Validation Against Objectives 28

8 Results and Discussion **29**
8.1 Operational KPIs from the Deployed System 29
8.2 Before-versus-After Comparison (Simulation) 30
8.3 Correctness Result 32

9 Benefits, Conclusion and Future Work **33**
9.1 Benefits and Real-World Applications 33
9.2 Conclusion 33
9.3 Project Links 34
9.4 Future Enhancements 34

References **36**

A Important Code Snippets **38**

List of Figures

3.1 System architecture of SmartDock 7
3.2 Context diagram (DFD level 0) 8
3.3 Level-1 data flow diagram 8
3.4 Use case diagram 9
3.5 Entity–relationship diagram 10
3.6 Booking state machine 10

5.1 Atomic capacity reservation algorithm 18

6.1 Entry point and manager overview 20
6.2 Yard schedule: door, carrier, PO/ASN, truck, driver, load and the next action for each truck 20
6.3 Dock configuration and carrier management 21
6.4 Analytics and KPIs 21
6.5 Scheduling rules and audit trail 22
6.6 Notifications and gate operations 22
6.7 Carrier dashboard and booking wizard (1) 23
6.8 Booking wizard (2): slot selection and shipment details 23
6.9 Booking wizard (3): confirmation and weekly-series result 24

8.1 Carrier reliability scores computed by the system 30
8.2 Utilisation by dock door 30
8.3 Simulated effect of appointment scheduling 31

CHAPTER 1

Introduction

1.1 Role of Information Technology in Modern Supply Chains

A supply chain is the network of organisations, facilities and activities that move a product from raw material to the final customer: sourcing, production, warehousing, transportation and distribution [1]. Competitive advantage increasingly depends not on individual firms but on how well this network is *coordinated* [2]. Information technology is the mechanism of that coordination. It provides:

- **Visibility** — a shared, real-time view of orders, inventory, shipments and capacity across partners, replacing phone calls and spreadsheets.
- **Synchronisation** — planning tools that align the activities of independent parties (suppliers, carriers, warehouses) to a common schedule.
- **Automation** — rules executed by software (approvals, alerts, exception handling) that reduce manual effort and errors.
- **Measurement** — data captured at every event that turns into key performance indicators (KPIs) such as on-time delivery, utilisation and lead time, enabling continuous improvement.

Typical supply-chain information systems include Enterprise Resource Planning (ERP), Warehouse Management Systems (WMS), Transportation Management Systems (TMS) and Yard Management Systems (YMS) [3]. Between the TMS (which plans *when* a truck travels) and the WMS (which plans *what* happens inside the building) lies a frequently neglected handover point: the **loading dock**. Dock appointment scheduling systems fill that gap by agreeing, in advance, the exact time and door at which each truck will be loaded or unloaded.

1.2 Problem Statement

Many warehouses and distribution centres — especially small and mid-sized ones — still manage inbound deliveries and outbound pickups by telephone, e-mail and whiteboards, or allow carriers to arrive whenever convenient. This leads to well-documented operational problems:

1. **Peaks and congestion.** Carriers prefer the same morning and afternoon hours. Trucks queue in the yard and on public roads while docks are idle at other times.

2. **Detention and wasted capacity.** Drivers wait unpaid or charge detention fees; dock labour cannot be planned against a known workload.
3. **Double-booking and mismatches.** Two carriers are promised the same door, or a refrigerated (reefer) or hazardous load is sent to a door that cannot handle it.
4. **No accountability.** Without arrival timestamps there is no objective record of which carriers are late or fail to show up, so carrier performance cannot be managed.
5. **No data for decisions.** Managers cannot answer basic questions such as "How utilised are our docks?", "Which hours are busiest?" or "What is our average truck turnaround time?"

Problem statement. Design and deploy a web-based system that lets warehouses publish dock capacity and lets carriers self-book compatible, conflict-free appointments, while capturing yard events to measure and improve dock utilisation, punctuality and turnaround — securely, for many warehouses and carriers on one platform.

1.3 Objectives

1. To build a **multi-tenant** platform where each warehouse controls its own docks, rules and approved carriers, and carriers book only where approved.
2. To provide a **configurable scheduling engine**: operating hours, slot length (15/30/60/90 minutes), turnover buffer, holidays (blackout dates) and dock maintenance windows.
3. To **match loads to docks** by movement direction (inbound/outbound), trailer type (dry van, reefer, flatbed, container) and hazmat certification.
4. To **guarantee** that the capacity of a dock slot is never exceeded, even under simultaneous requests on multiple servers.
5. To support realistic carrier workflows: **recurring** weekly appointments, **rescheduling**, and an automatic **waitlist**.
6. To digitise **yard execution** (gate check-in, dock start, completion) and automatically detect **late arrivals and no-shows**.
7. To compute a **carrier reliability score** and operational **KPIs** with dashboards, heatmaps and CSV/PDF reports.
8. To keep a complete **audit trail** and send **notifications** (in-app, e-mail, SMS) for every important event.
9. To deploy the system on the **cloud** and validate it with automated tests and a simulation study.

1.4 Scope and Limitations

Scope

- Users: warehouse/DC managers (one per warehouse), carriers (transport companies) and a platform super administrator.
- Functions: site and dock configuration, appointment booking and lifecycle, waitlist, yard events, notifications, audit, analytics and exports, as listed in the objectives.
- Deployment: a single cloud web service (Render) with a managed MongoDB Atlas database, accessible from any browser.
- Demo mode: a realistic dataset (two DCs, five typed docks at the Chennai DC, six carriers, about five weeks of history) for evaluation and presentation.

Limitations

- **No WMS/TMS integration.** Purchase orders and advance shipping notices (ASN) are entered as references; they are not fetched from an ERP.
- **Manual gate events.** Check-in and completion are recorded by staff clicking buttons; there is no automatic number-plate recognition or IoT sensor input.
- **Fixed-length slots.** All appointments at a site have the same length; load-dependent durations (e.g. 90 minutes for a full container) are not yet modelled.
- **Super administrator.** The role exists and is protected, but it has no dedicated user interface yet.
- **Simulated evaluation.** The dataset and the before/after comparison are generated or simulated, not measured at a real warehouse.
- **Free-tier hosting.** The free cloud instance sleeps when idle (about 50 s cold start) and runs on a small CPU, which affects response times.

CHAPTER 2

Background and Literature Survey

2.1 Background: Docks, Yards and Appointments

A **dock door** is a bay where a truck is loaded or unloaded. **Inbound** (receiving) docks accept deliveries from suppliers; **outbound** (shipping) docks dispatch customer orders. The **yard** is the area where trucks wait before and after docking. Warehouse operations research identifies receiving and shipping as critical, labour-intensive stages whose performance strongly affects the whole facility [4].

Docks are heterogeneous. A *reefer* trailer carries temperature-controlled goods and must go to a door with cold-chain facilities; a *flatbed* needs side or crane access; *hazardous materials* require certified bays with spill containment. Each door can also serve a limited number of trucks per time period.

A **dock appointment** (also called a *truck appointment* or *time-slot booking*) is a reservation of a specific door for a specific time window. The key performance indicators of dock operations include:

- **Dock utilisation** — booked truck-slots divided by offered truck-slots.
- **On-time arrival rate** — share of trucks arriving within an agreed tolerance of their slot.
- **No-show rate** — confirmed appointments where the truck never arrived.
- **Turnaround (gate-to-gate) time** — time from gate check-in to departure; split into *yard wait* (gate to door) and *dock time* (loading or unloading).

2.2 Literature Survey

Truck appointment systems at ports. The most studied application of appointment scheduling in freight is the marine container terminal. Giuliano and O'Brien [6] evaluated the gate appointment system introduced at the Ports of Los Angeles and Long Beach and found that its benefits depended heavily on how it was designed and enforced; a system that does not actually control arrivals delivers little congestion relief. Huynh and Walton [7] modelled how capping the number of trucks per time window reduces truck turn times at terminals and showed the trade-off between the cap and terminal utilisation. Zehendner and Feillet [8] showed with an optimisation model that a truck appointment system improves overall service quality at a multimodal terminal by smoothing truck arrivals. Phan and Kim [9] studied how arrival times can be negotiated between trucking companies and a terminal to reduce waiting for both sides.

Warehouse and cross-dock scheduling. Gu, Goetschalckx and McGinnis [4] review warehouse operations and highlight receiving and shipping as planning problems in their own right. Boysen and Fliedner [5] classify truck scheduling problems at cross-docks, where the assignment of trucks to doors and time slots determines the facility's throughput.

Information systems. Supply-chain textbooks [1, 2, 3] position appointment scheduling within the wider trend of collaborative, information-driven logistics, in which partners share capacity information instead of competing for it.

Software reliability. A less discussed but practical issue is correctness under concurrency. Kleppmann [11] describes *write skew*: two transactions both read that a resource is free and both reserve it. A booking system that is vulnerable to this anomaly can double-book a dock exactly when demand is highest. MongoDB's multi-document transactions [12] can prevent it only if the transactions are made to conflict on a common document.

Simulation. Discrete-event simulation is a standard method for comparing queueing systems under alternative policies before implementation [10]; it is used in this project to estimate the effect of appointments on yard waiting.

2.3 Existing Systems

Table 2.1 compares common approaches to dock scheduling. The commercial categories are described at a general level from their public positioning; individual products differ.

Table 2.1: Comparison of existing approaches with SmartDock

Criterion	**Phone / e-mail / spreadsheet**	**Port truck appoint-ment systems**	**Commercial dock-scheduling SaaS**	**WMS / YMS suite modules**	**SmartDock (this project)**
Carrier self-service	No	Yes	Yes	Partial	Yes
Capacity enforced	No (manual)	Yes	Yes	Yes	Yes, atomically across servers
Dock-type / hazmat matching	Manual	Limited (container focus)	Varies	Yes	Yes
Waitlist, recurring, reschedule	No	Varies	Varies	Varies	Yes
Carrier scorecard & KPIs	No	Terminal KPIs	Yes	Yes	Yes
Audit trail	No	Yes	Yes	Yes	Yes
Cost / effort	Low cost, high effort	Operated by ports	Subscription	High licence and integration cost	Free, open source
Customisable	—	No	Limited	Limited	Fully (own code)

2.4 Gap in Existing Systems

- Small and mid-sized warehouses are left between *manual* methods, which do not scale and produce no data, and *enterprise* suites, which are expensive and require long integration projects.
- Port appointment systems are designed for container terminals, not for multi-tenant warehouses with heterogeneous docks.
- Many simple booking tools treat a slot as a calendar entry and do not handle *dock compatibility*, *per-slot capacity* or *concurrency*, so they can double-book at peak times.
- Booking is often separated from execution: if gate events are not captured, KPIs such as on-time rate and turnaround cannot be computed, and carriers cannot be held accountable.

2.5 Proposed Solution

SmartDock closes these gaps with a single, lightweight, cloud-hosted application that:

1. lets every warehouse publish its own dock capacity and rules, and lets carriers request access per warehouse;
2. offers only *compatible* docks and *valid* slots, and enforces slot capacity with a concurrency-safe reservation algorithm;
3. covers the full appointment lifecycle — request, approval, rescheduling, recurring series, waitlist, gate check-in, dock start, completion, no-show;
4. turns the captured events into KPIs, a carrier reliability score and exportable reports;
5. records every action in an audit trail and informs both parties through notifications;
6. is free, open source and deployable in minutes with one configuration file.

CHAPTER 3

System Design

3.1 System Architecture

SmartDock follows a classic three-tier, layered architecture (Figure 3.1). The **presentation tier** is a single-page React application running in the user's browser. The **application tier** is a Node.js/Express REST API that also serves the built React files, so the whole system is one deployable web service. Requests pass through security middleware (rate limiting, JWT authentication, role and tenant checks) before reaching controllers, which delegate business logic to reusable services. The **data tier** is a MongoDB Atlas replica set, which provides the multi-document transactions required by the booking guarantee. Optional external providers deliver e-mail (Resend) and SMS (Twilio). Code pushed to GitHub is built and deployed automatically by Render.

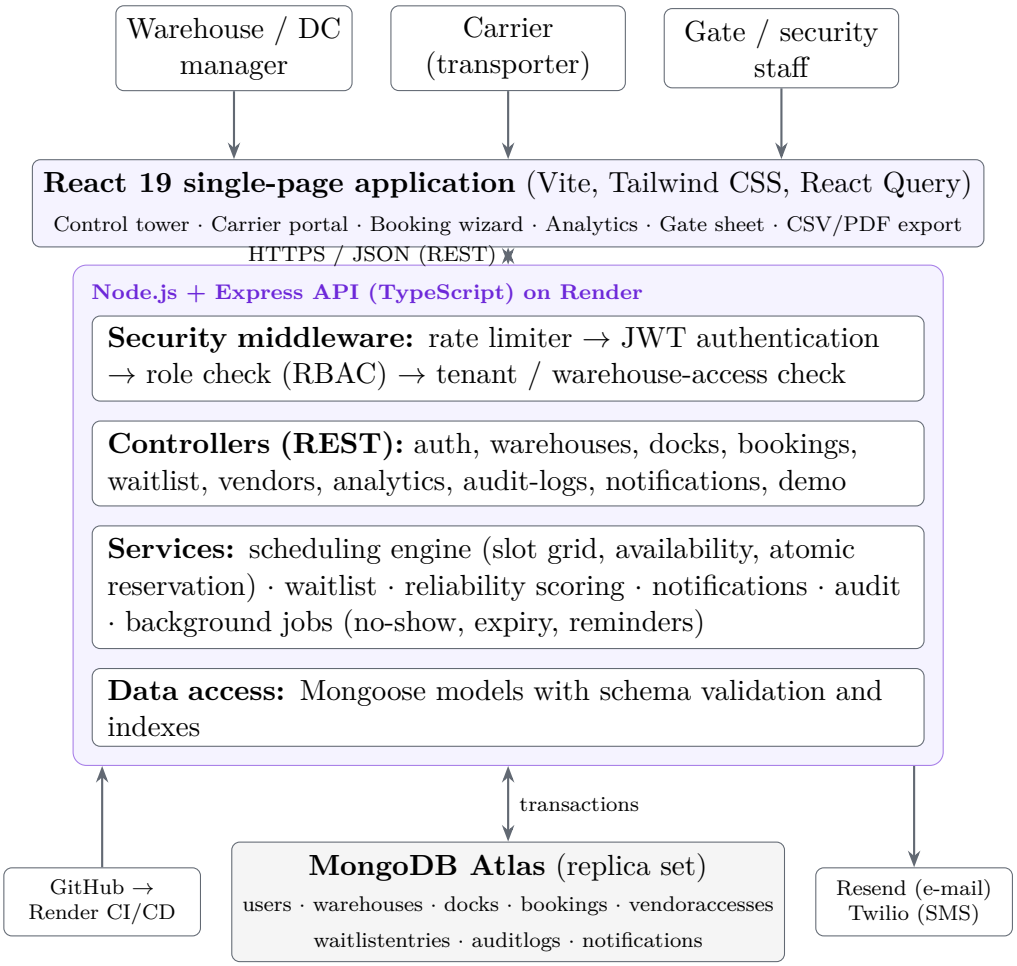


Figure 3.1: System architecture of SmartDock

3.2 Data Flow Diagrams

Figure 3.2 shows the context (level-0) diagram: SmartDock exchanges information with three human actors and the notification providers. Figure 3.3 decomposes the system into six processes and five data stores.

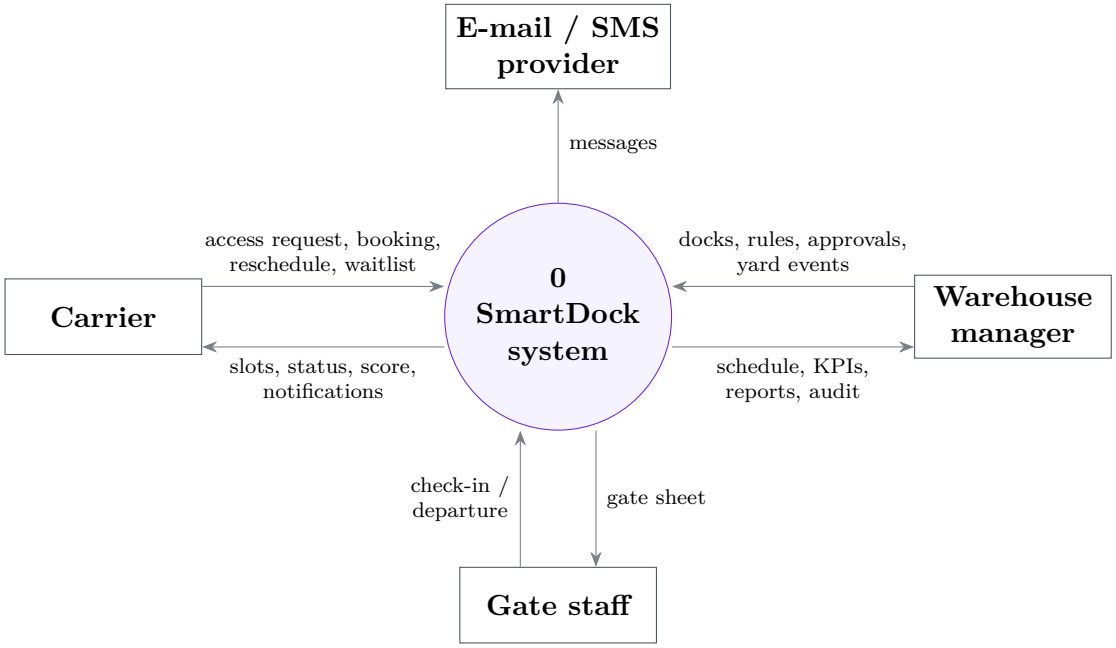


Figure 3.2: Context diagram (DFD level 0)

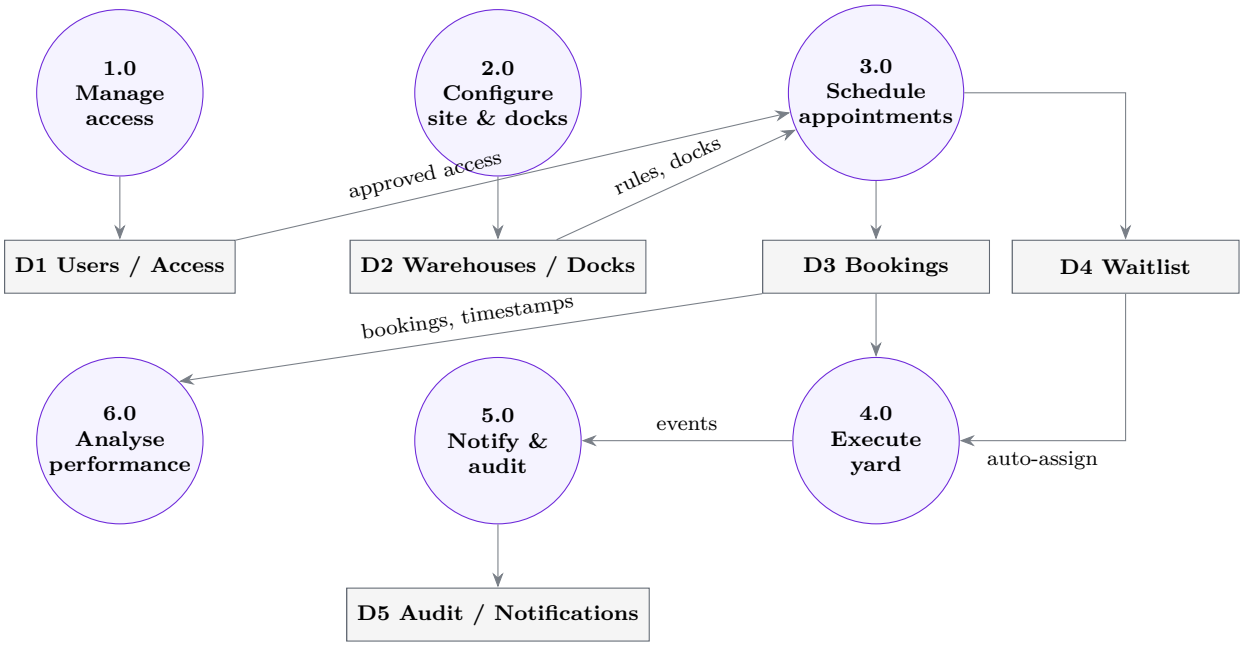


Figure 3.3: Level-1 data flow diagram

3.3 Use Case Diagram

Figure 3.4 shows what each actor can do. The scheduler represents the automated background jobs.

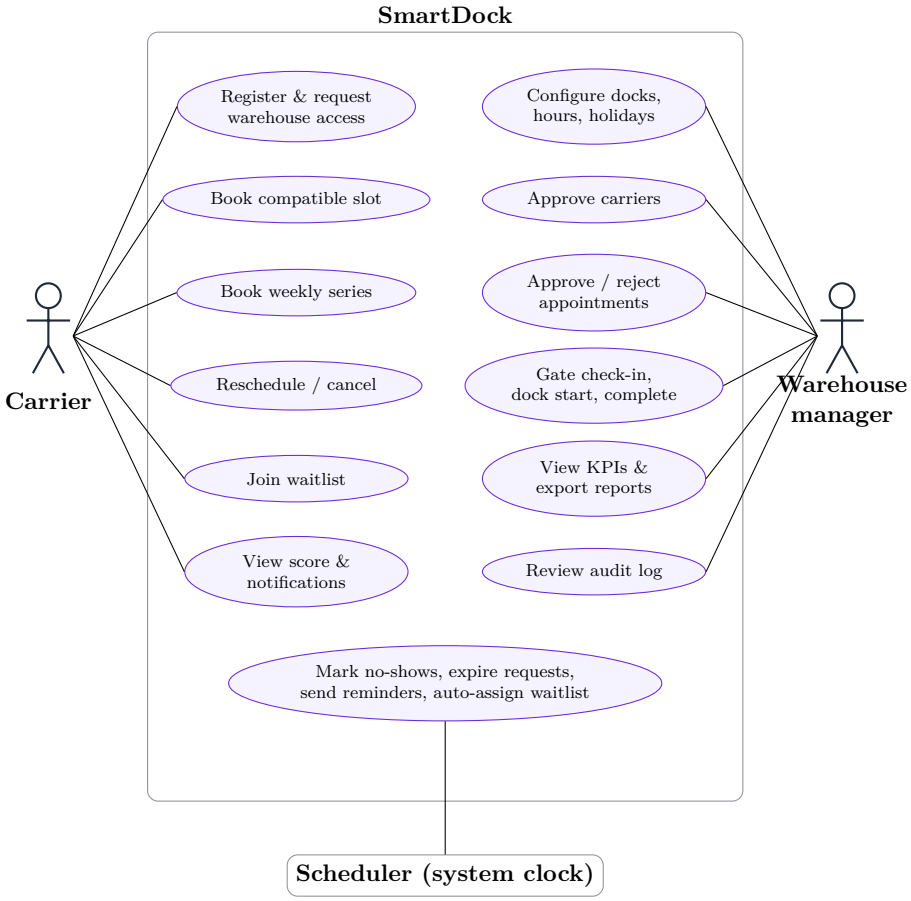


Figure 3.4: Use case diagram

3.4 Entity–Relationship Diagram

Figure 3.5 shows the main collections and their relationships. MongoDB stores them as documents; references are ObjectIds and are enforced in application code.

3.5 Appointment Lifecycle

Each booking follows a finite-state machine (Figure 3.6). Terminal states can never be revived, so capacity released by a cancellation cannot be silently re-occupied.

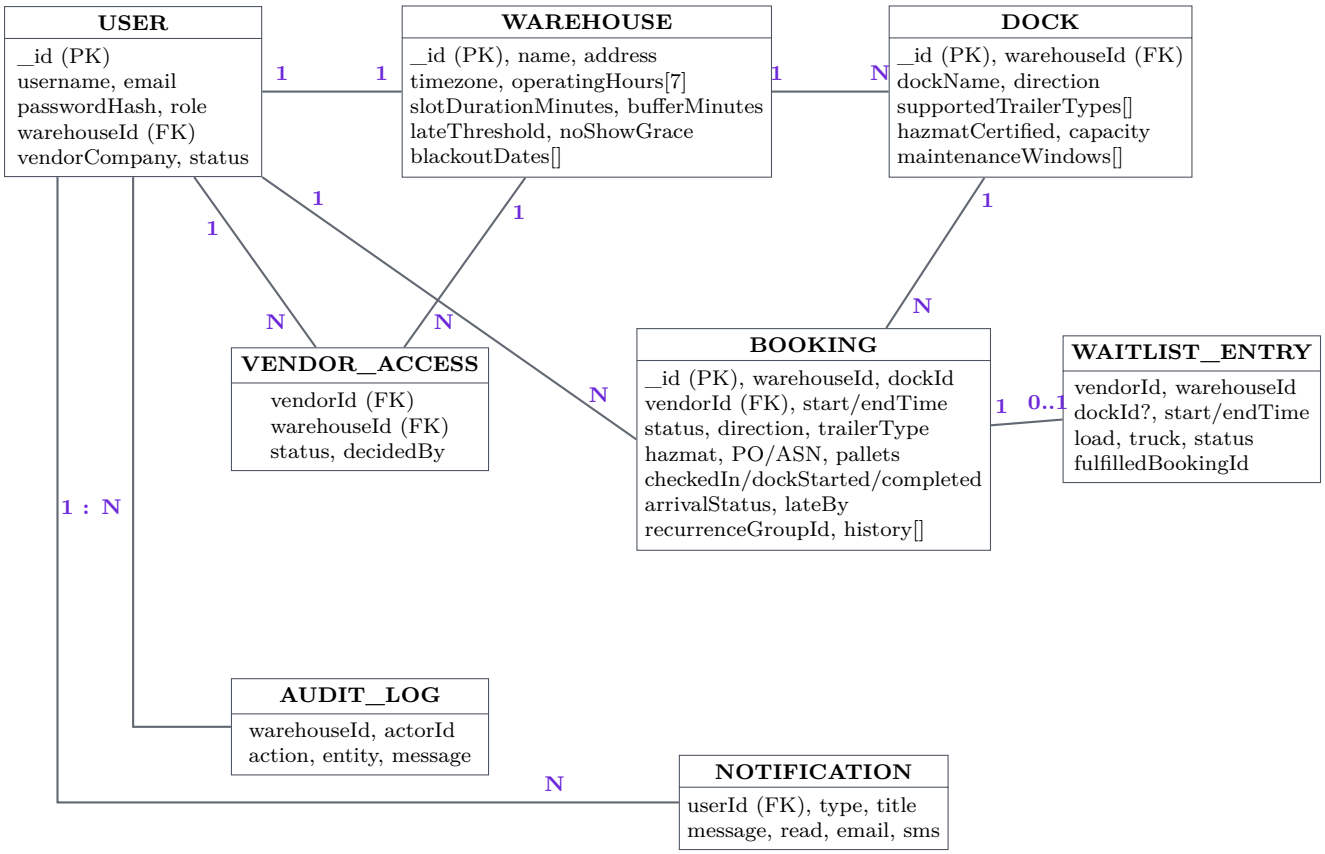


Figure 3.5: Entity–relationship diagram

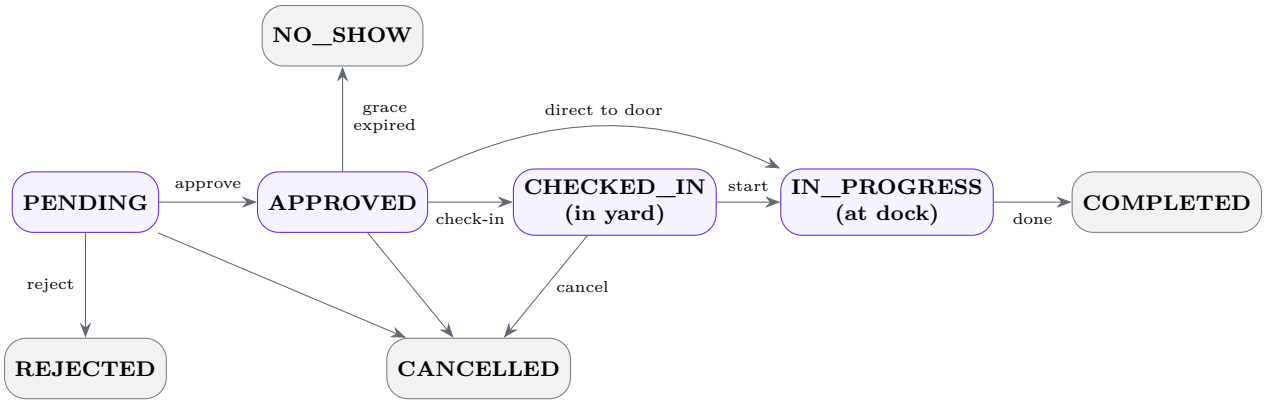


Figure 3.6: Booking state machine

3.6 Technology Stack

Table 3.1: Technology stack

Layer	Technologies and purpose
Language	TypeScript (strict mode) on both client and server
Frontend	React 19 [14], Vite 8, Tailwind CSS 4, TanStack React Query (data fetching, polling), React Router, React Hook Form + Zod (forms), date-fns, jsPDF + AutoTable (PDF export)
Backend	Node.js 22, Express 4 [15], Zod (request validation), JSON Web Tokens, bcrypt password hashing, express-rate-limit, Winston logging
Database	MongoDB Atlas (managed replica set) with Mongoose 8 ODM; multi-document transactions, compound indexes, aggregation pipelines
APIs	REST/JSON API (about 40 endpoints); Resend e-mail API and Twilio SMS API (optional)
Cloud & DevOps	Render web service [16] (Blueprint `render.yaml`, auto-deploy from GitHub), MongoDB Atlas free tier, GitHub for version control
Testing	End-to-end verification script with `mongodb-memory-server` (in-memory replica set) and two concurrent API instances; tenant-isolation unit test
Tools	Visual Studio Code, Git, npm, oxlint, Python + matplotlib (analysis and simulation), LaTeX (this report)

CHAPTER 4

Modules Description

SmartDock is organised into ten functional modules. Table 4.1 summarises them; the sections that follow describe each in supply-chain terms.

Table 4.1: Functional modules and their main users

Module	Responsibility	Used by
1. Identity & access	Registration, login, roles, per-warehouse carrier approval	All
2. Site & dock configuration	Docks, equipment, capacity, hours, slot rules, holidays, maintenance	Manager
3. Scheduling engine	Slot grid, availability, load–dock matching, atomic reservation	System
4. Appointment booking	Booking wizard, recurring series, rescheduling, cancellation	Carrier, manager
5. Waitlist	Queue for full slots, automatic FIFO assignment	Carrier, system
6. Yard execution	Approvals, gate check-in, dock start, completion, no-shows	Manager, gate
7. Carrier performance	Punctuality tracking, reliability score and tier	Manager, carrier
8. Analytics & reporting	KPIs, charts, heatmap, CSV/PDF export, gate sheet	Manager
9. Notifications	In-app centre, e-mail, SMS, reminders	All
10. Audit trail	Persistent, filterable log of every action	Manager

4.1 Identity and Access Management

Three roles exist: **warehouse manager**, **carrier** and **super administrator**. A manager registers together with the warehouse they run. A carrier registers with its company name and selects the warehouses it wants to serve; each selection creates a *vendor-access request*. A warehouse's manager sees only requests for that warehouse and can approve, reject or later revoke them. A carrier can see availability and book *only* at warehouses that approved it, and the check is repeated on every request so a revocation takes effect immediately. Super administrators cannot be created through the public sign-up form; they are created on the server with a command-line script. Passwords are stored as bcrypt hashes and sessions use signed JSON Web Tokens.

4.2 Site and Dock Configuration

The manager models the physical facility:

- **Dock doors** — name, movement direction (inbound, outbound or both), supported trailer types (dry van, reefer, flatbed, container), hazmat certification and capacity (trucks per slot).
- **Operating hours** — per weekday, with an open/closed flag.
- **Appointment rules** — slot length (15, 30, 60 or 90 minutes), turnover buffer between consecutive slots, late-arrival threshold and no-show grace period.
- **Blackout dates** — public holidays, stock-takes and shutdowns on which the site is closed.
- **Maintenance windows** — planned downtime of a single door; the system lists the existing appointments that overlap so they can be moved.

4.3 Scheduling Engine

The engine converts the rules into concrete, bookable slots for any date, in the warehouse's own time zone. For each active dock that is compatible with the requested load it reports, per slot, the remaining capacity and the reason a slot is unavailable (*full*, *maintenance* or *past*). It also validates every booking request against the same grid and performs the atomic capacity reservation described in Chapter 5.

4.4 Appointment Booking

Carriers book through a four-step wizard: (1) site and load details — inbound or outbound, trailer type, hazmat; (2) date and slot, showing only compatible docks; (3) shipment details — truck number, driver name and mobile, PO/ASN reference, pallet count, cargo; (4) confirmation. A booking may be made as a **weekly recurring series** of 2–12 weeks; each week is reserved independently and weeks that are closed or full are reported back rather than failing the whole series. Carriers and managers can **reschedule** an appointment to another slot or door; the history of moves is stored, and a carrier's move of a confirmed appointment returns it to the manager for re-approval. An entire series can be cancelled in one action.

4.5 Waitlist

If the desired slot is full, the carrier can join its **waitlist**, for a specific door or for any compatible door. Whenever capacity is released — by a cancellation, a rejection or a

reschedule — the system automatically converts the oldest compatible waiting entry into a booking (first-in, first-out), marks the entry as fulfilled and notifies both sides. Entries whose slot has passed expire automatically.

4.6 Yard Execution

The *Yard schedule* screen is the manager's daily control tower. For the selected day it lists every truck with its door, carrier, driver, load and references, and offers the next valid action: *Approve/Reject* for requests, *Gate check-in* when the truck arrives, *Start at dock* when loading begins, and *Complete & depart*. The system timestamps each event, classifies the arrival as *on time* or *late* against the warehouse threshold, and records the delay in minutes. Confirmed trucks that do not arrive within the grace period are marked *no-show* automatically; requests that were never approved expire when their slot starts. A printable *gate sheet* gives security staff the day's expected trucks with columns for gate times, seal numbers and signatures.

4.7 Carrier Performance

Every arrival and no-show feeds the carrier's **reliability score**:

$$\text{Reliability} = \frac{\text{on-time arrivals} + 0.5 \times \text{late arrivals}}{\text{arrivals} + \text{no-shows}} \times 100 \tag{4.1}$$

Scores map to tiers: **A** (≥ 90, excellent), **B** (≥ 75, good), **C** (≥ 50, needs improvement) and **D** (below 50, poor). Managers see the score and tier when approving a new carrier and in a scorecard table; carriers see their own score on their dashboard, which creates an incentive to arrive on time.

4.8 Analytics and Reporting

For any date range, the analytics module reports: total appointments and daily average; **dock utilisation** (booked truck-slots divided by offered truck-slots, excluding closures and maintenance); on-time, no-show and cancellation rates; average **turnaround**, **yard wait** and **dock time**; pallets handled; a daily volume chart; a weekday $\times$ hour **demand heatmap**; the busiest hours; utilisation per door; the inbound/outbound and equipment mix; and a carrier performance table. Results can be exported as CSV or as a formatted PDF KPI report.

4.9 Notifications

Each event creates an in-app notification for the relevant party (shown under a bell icon with an unread counter), for example: new request, approval, rejection, rescheduling, cancellation, waitlist assignment, no-show and new carrier access request. About 24 hours before a confirmed appointment a reminder is sent to the carrier and, when SMS is configured, to the driver's mobile. E-mail (Resend) and SMS (Twilio) are activated simply by adding API keys; delivery status is stored with each notification.

4.10 Audit Trail

Every change — bookings and status changes, reschedules, waitlist events, dock and rule changes, holidays, maintenance and carrier-access decisions — is written to an audit collection with the actor, role, time, action and a human-readable message. Automated actions are attributed to "SmartDock automation". Managers can filter, search, page through and export the log for their own warehouse only, which supports dispute resolution (for example, detention claims) and compliance.

CHAPTER 5

Implementation Details

This chapter explains the key algorithms. Complete source code is available in the GitHub repository; the most important excerpts are reproduced in Appendix A.

5.1 Project Structure

- `client/` — React application: `pages/admin` (control-tower tabs and gate sheet), `pages/vendor` (carrier portal and booking wizard), `components` (slot picker, reschedule dialog, notification bell, UI kit), `lib` (formatting and CSV/PDF export).
- `server/src/models` — Mongoose schemas: User, Warehouse, Dock, Booking, VendorAccess, WaitlistEntry, AuditLog, Notification.
- `server/src/services` — business logic: `scheduling`, `waitlist`, `reliability`, `notifications`, `audit`, `jobs`, `seed`.
- `server/src/controllers` and `routes` — the REST API; `middleware` — authentication, roles and tenant access.
- `render.yaml` — infrastructure-as-code definition of the cloud deployment.

5.2 Time-Zone-Correct Slot Generation

Warehouses operate in local time, but all instants are stored in UTC so that sites in different zones and daylight-saving transitions are handled correctly. Given a local date d, the engine:

1. returns *closed* if d is a blackout date or the weekday is closed;
2. converts opening time o and closing time c to minutes and generates slots

$$s_k = o + k\,(L + B), \qquad e_k = s_k + L, \qquad \text{while } e_k \le c,$$

   where L is the slot length and B the turnover buffer (e.g. $L = 30$, $B = 15$ gives 08:00–08:30, 08:45–09:15, …);
3. converts each local start and end to an exact UTC instant using the IANA time-zone database. Because a local time's UTC offset depends on the date, the conversion starts from a guess and corrects it by the observed difference until it converges (at most three iterations; Listing A.2).

For availability, each slot of each compatible door is then checked against existing active bookings (overlap test $a_s < b_e \wedge a_e > b_s$), maintenance windows and the current time.

5.3 Load–Dock Compatibility

A load is described by its direction, trailer type and hazmat flag. A dock is compatible if (i) its direction is *both* or equals the load's direction, (ii) its supported trailer types include the load's type, and (iii) it is hazmat-certified whenever the load is hazardous. Incompatible doors are never shown to the carrier, and the same rule is re-checked on the server when a booking is submitted (Listing A.3).

5.4 Atomic Capacity Reservation

The central correctness requirement is that a slot with capacity C never holds more than C active bookings. A naive implementation "count overlapping bookings, then insert if fewer than C" is unsafe: two requests can both count $C - 1$ and both insert — the *write-skew* anomaly [11]. Running both steps inside a database transaction is *not* sufficient by itself, because each transaction reads its own snapshot and they write different documents, so the database sees no conflict.

SmartDock therefore uses two layers (Listing A.4):

1. **In-process lock per dock.** Requests for the same door handled by one server process are queued and executed one at a time.
2. **Dock write lock inside a MongoDB transaction.** Each reservation transaction first increments a counter field on the *dock document*, then counts overlapping active bookings and inserts only if the count is below capacity. Because every concurrent reservation for the same door now writes the same document, MongoDB detects a write conflict, aborts all but one transaction and the driver retries them; the retries see the committed booking and are rejected with HTTP 409 if the slot is full. This holds across any number of server instances.

The same routine is reused for new bookings, each week of a recurring series, rescheduling (excluding the booking being moved) and waitlist assignment. If transactions are unavailable (a standalone development database), the system falls back to the in-process lock and logs a warning.

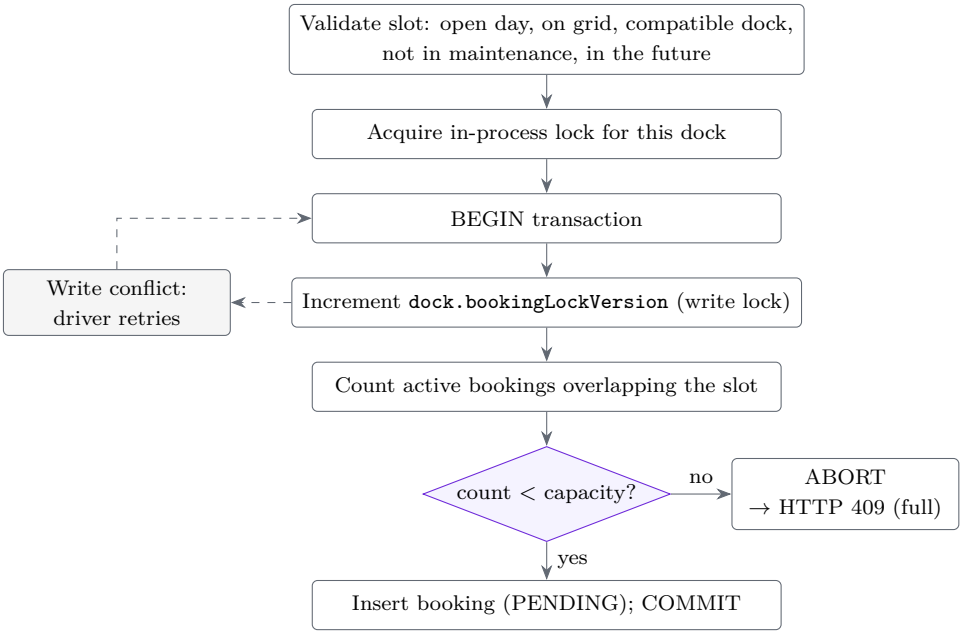


Figure 5.1: Atomic capacity reservation algorithm

5.5 Recurring Bookings and Rescheduling

For a weekly series of n weeks, the local start time of the first slot (e.g. 09:00) is kept and, for each week k, the slot with the same local start on date $d + 7k$ is looked up in that day's grid. This keeps the appointment at 09:00 local time even across daylight-saving changes. Each week is validated and reserved separately; failures are collected as "skipped weeks". Rescheduling validates the new slot with the same rules, reserves capacity while excluding the booking itself, records the move in the booking's history and finally offers the released old slot to the waitlist.

5.6 Waitlist Auto-Assignment

When capacity is released on a door and slot, the waitlist service selects `WAITING` entries for exactly that slot (for that door or "any door"), ordered by creation time. For each entry it re-checks dock compatibility and the carrier's access, then calls the reservation routine. Inside the same transaction it marks the entry `FULFILLED`, so an entry can never be assigned twice. The loop stops at the first "slot full" result (Listing A.7).

5.7 Arrival Classification and Background Jobs

At gate check-in the delay is computed as $\Delta = t_{\text{check-in}} - t_{\text{slot start}}$ in minutes; the arrival is *late* if Δ exceeds the warehouse's threshold (default 15 minutes) and *on time* otherwise, including early arrivals. A background job runs every minute and (i) marks confirmed bookings as `NO_SHOW` when the grace period after the slot start has elapsed without check-

in, (ii) expires pending requests whose slot has started, (iii) sends reminders for confirmed appointments in the next 24 hours, and (iv) expires waitlist entries for past slots. Every update is conditional on the previous state, so the job is safe to run on several instances at once (Listing A.9).

5.8 Security Measures

- **Authentication**: bcrypt-hashed passwords, signed JWTs with expiry; the client stores the session and logs out automatically on token expiry.
- **Authorisation**: role checks on every route and a *tenant* check that pins managers to their own warehouse and carriers to warehouses that approved them (Listing A.10); identifiers are taken from the verified token, never trusted from the request body. This addresses broken access control, the top risk in the OWASP Top 10 [13].
- **Validation**: every request body is validated with Zod schemas; database schemas validate again before writing.
- **Abuse protection**: rate limiting on the API; super administrators cannot self-register.

5.9 Deployment

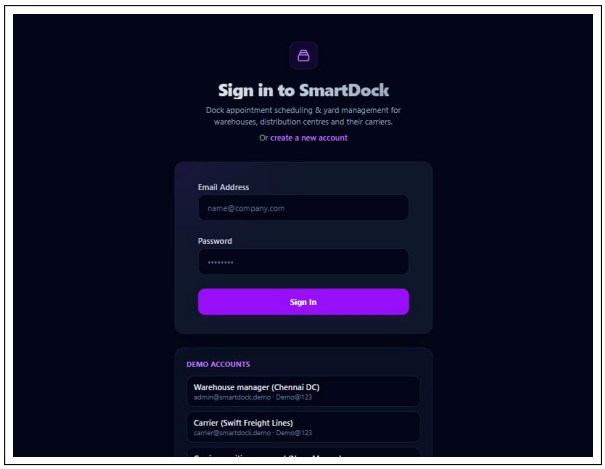
The Express server serves both the API and the compiled React files, so the system is a single Render web service. The `render.yaml` Blueprint declares the build command (build client, then server), start command, health check, auto-deploy on every push to the `main` branch and the environment variables (database URI, generated JWT secret, optional e-mail/SMS keys and demo mode). The database is a MongoDB Atlas free-tier replica set in the Mumbai region. In demo mode the server seeds realistic data on first start and offers a "Reset demo data" action.

CHAPTER 6

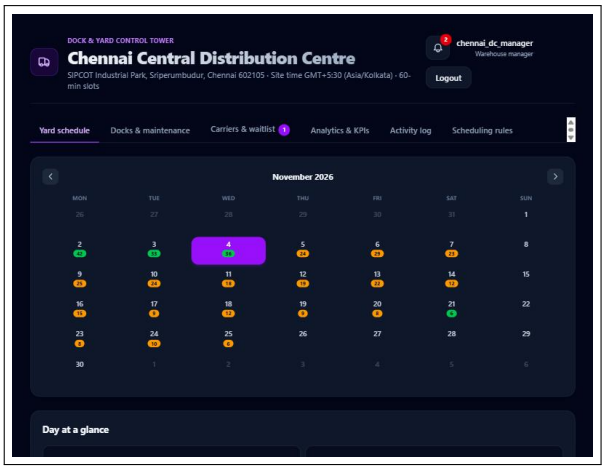
Screenshots and Live Demo

The system is live at `https://smartdock.onrender.com`. The screenshots below were captured from the running application with the demo dataset (Chennai Central Distribution Centre, five dock doors, six carriers). Demo logins: `admin@smartdock.demo` (warehouse manager) and `carrier@smartdock.demo` (carrier), password `Demo@123`.

Warehouse manager (control tower). Figures 6.1–6.6.



(a) Login with one-click demo accounts



(b) Control tower and appointment calendar

Figure 6.1: Entry point and manager overview

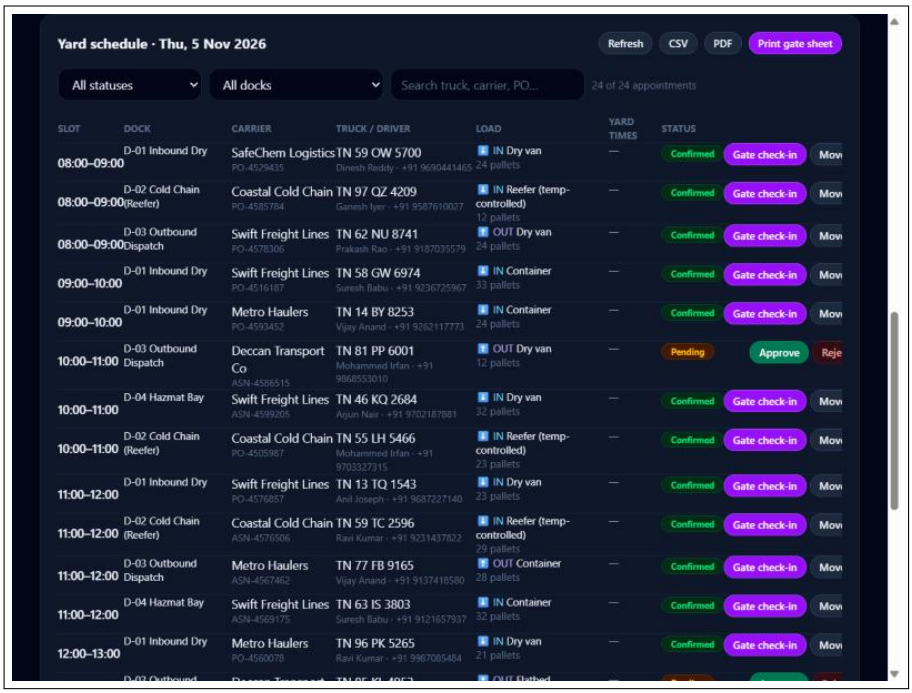
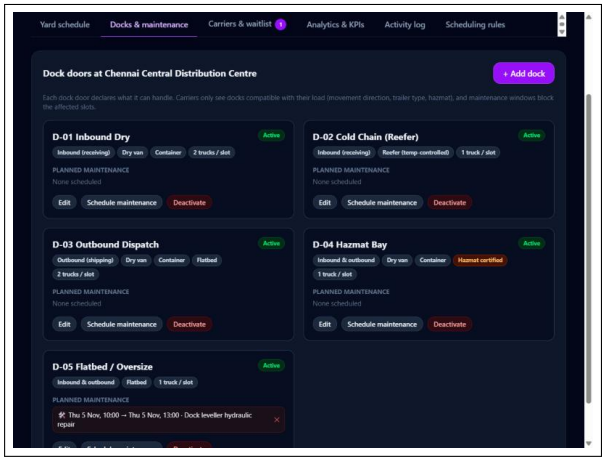
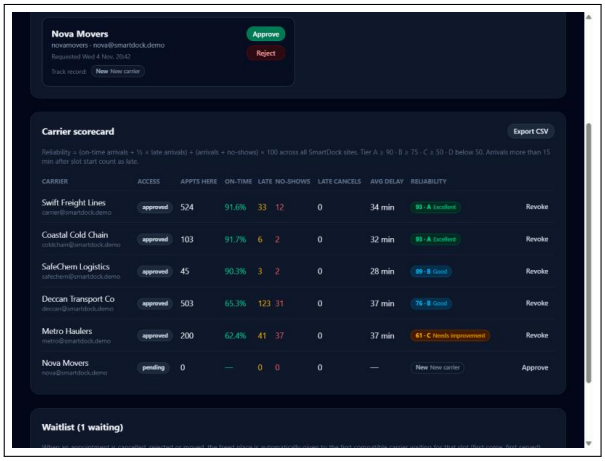


Figure 6.2: Yard schedule: door, carrier, PO/ASN, truck, driver, load and the next action for each truck

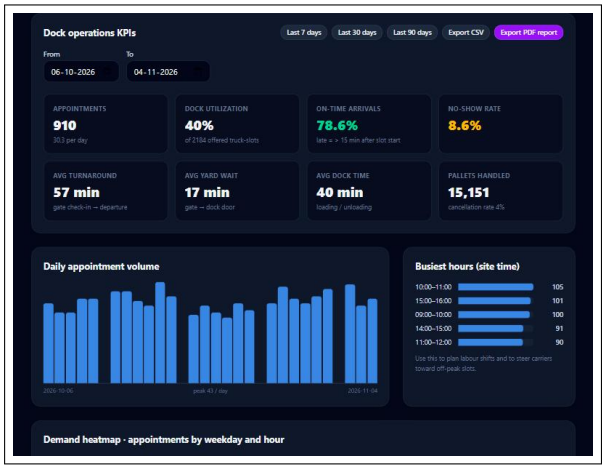


(a) Dock doors: direction, trailer types, hazmat, capacity and a maintenance window

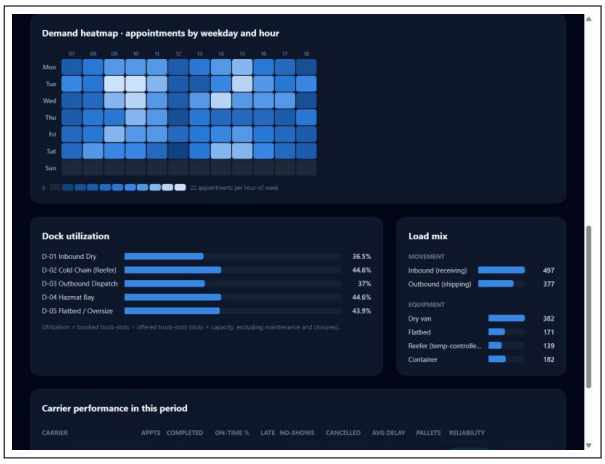


(b) Carrier access request and scorecard with reliability tiers

Figure 6.3: Dock configuration and carrier management

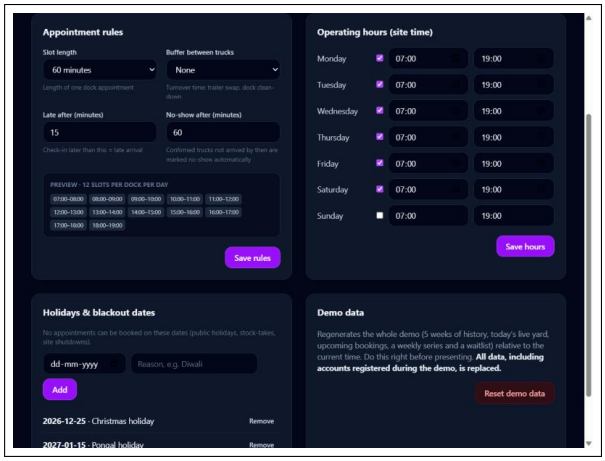


(a) KPI cards, daily volume and busiest hours

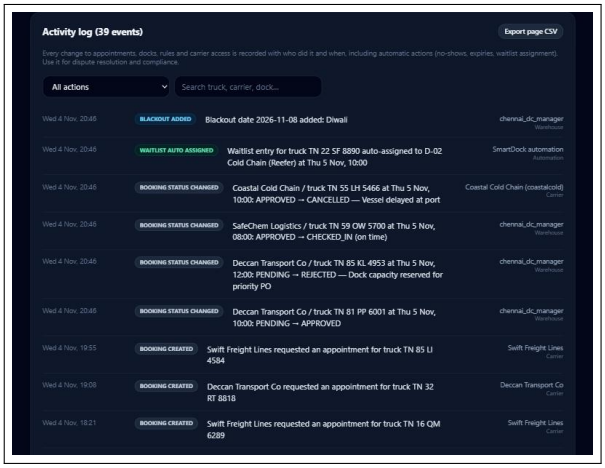


(b) Demand heatmap, utilisation per door and load mix

Figure 6.4: Analytics and KPIs

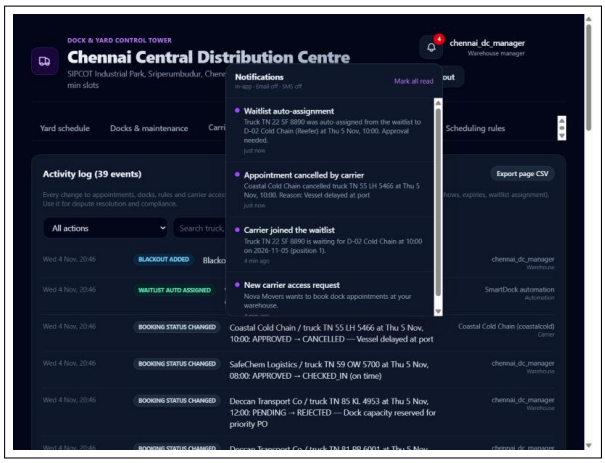


(a) Scheduling rules with live slot preview, hours and holidays

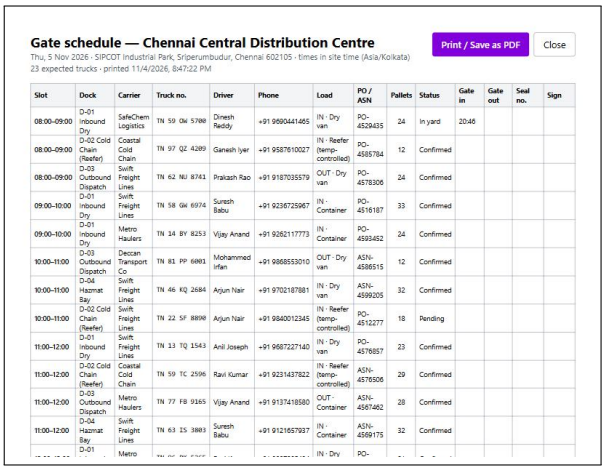


(b) Activity log: holiday, waitlist auto-assignment, cancellation, check-in, rejection

Figure 6.5: Scheduling rules and audit trail



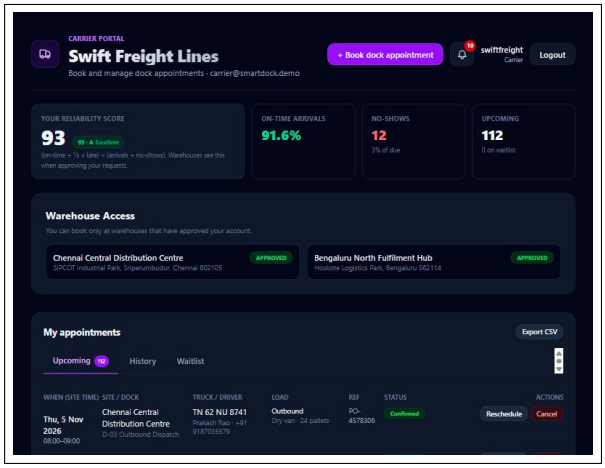
(a) Notification centre



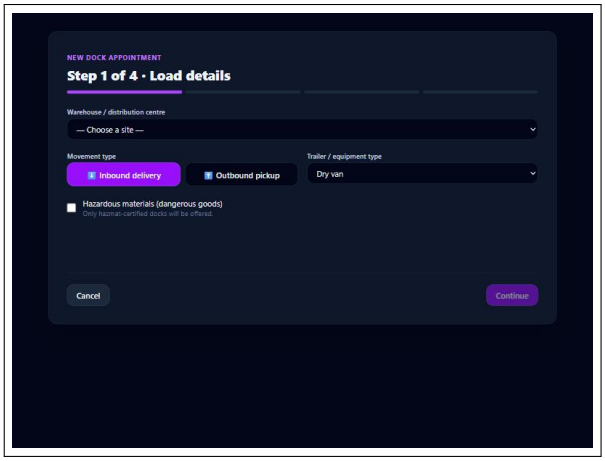
(b) Printable daily gate sheet for security staff

Figure 6.6: Notifications and gate operations

Carrier portal and booking wizard. Figures 6.7–6.9.

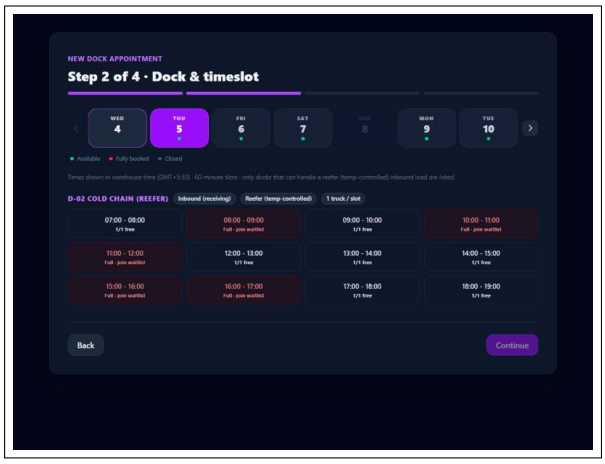


(a) Carrier dashboard: reliability score, access status, appointments

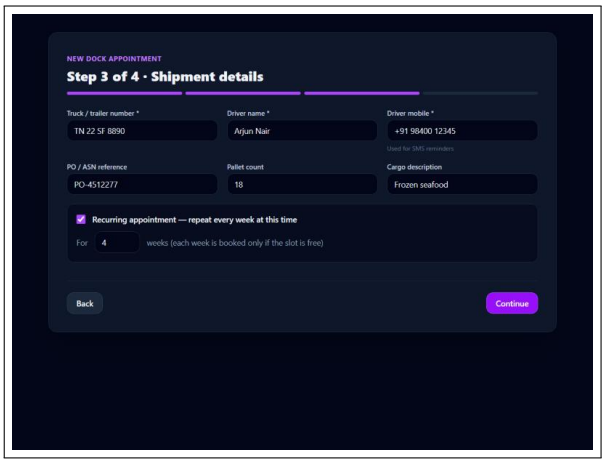


(b) Step 1: site, movement, trailer type, hazmat

Figure 6.7: Carrier dashboard and booking wizard (1)

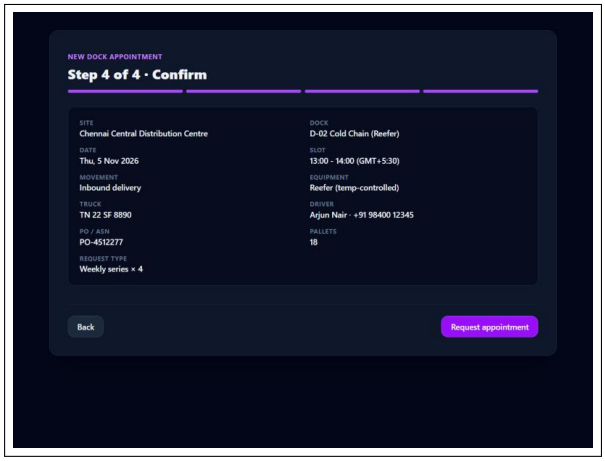


(a) Step 2: only the reefer dock is offered; full slots offer the waitlist

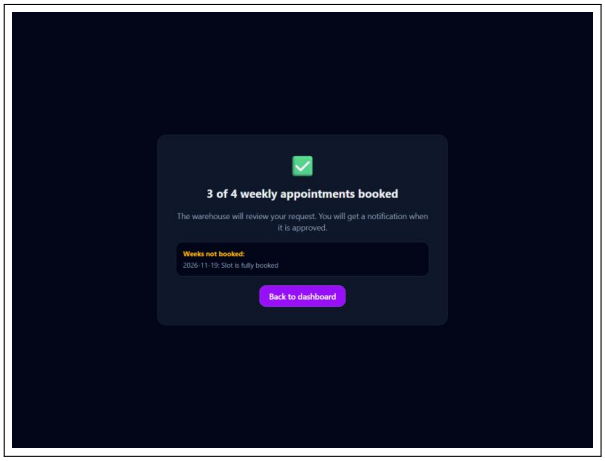


(b) Step 3: truck, driver, PO/ASN, pallets and weekly recurrence

Figure 6.8: Booking wizard (2): slot selection and shipment details



(a) Step 4: confirmation summary



(b) Result: 3 of 4 weekly appointments booked, one full week skipped

Figure 6.9: Booking wizard (3): confirmation and weekly-series result

CHAPTER 7

Testing and Validation

7.1 Testing Strategy

- **Static checks:** TypeScript strict type-checking of client and server on every build; `oxlint` for the client.
- **Unit test:** tenant isolation of dock updates, run against a mocked database.
- **End-to-end test suite** (`npm run verify:critical`): starts a throw-away in-memory MongoDB *replica set* and **two** independent API server processes, then drives the real HTTP API through 58 checks covering security, concurrency, scheduling rules and business workflows. Running two servers makes the concurrency test meaningful for a multi-instance cloud deployment.
- **Manual UI testing:** every screen was exercised in the browser with the demo data (Chapter 6).
- **Live smoke test:** after deployment, logins, analytics, booking, approval, gate check-in, cancellation and the audit log were verified against the production URL and database.

7.2 Test Cases

Table 7.1 lists representative test cases from the 58 automated checks. All passed.

Table 7.1: Representative automated test cases

ID	Test case	Expected result	Status
Security and multi-tenancy			
T01	Register a SUPER_ADMIN through the public API	Rejected (400)	Pass
T02	Unapproved carrier requests slot availability	Forbidden (403)	Pass
T03	Manager of warehouse B lists or approves a request for warehouse A	Not visible; approval returns 404	Pass
T04	Carrier approved at A tries to book at B	Forbidden (403)	Pass
T05	Manager B changes a booking of warehouse A	Forbidden (403)	Pass

ID	Test case	Expected result	Status
T06	Manager A reads warehouse B's audit log	B's events not returned	Pass
Capacity and concurrency			
T07	40 simultaneous bookings, capacity-1 slot, two servers	Exactly 1 created, 39 rejected (409)	Pass
T08	40 simultaneous bookings, capacity-3 slot, two servers	Exactly 3 created, 37 rejected (409)	Pass
T09	Availability after T08	Slot shows 0 remaining	Pass
T10	Bookings use database transactions	No fallback warning logged	Pass
Scheduling rules			
T11	Booking not aligned to the slot grid (09:15)	Rejected (400)	Pass
T12	Booking outside operating hours (03:00)	Rejected (400)	Pass
T13	Set 30-min slots + 15-min buffer	Grid 08:00–08:30, 08:45–09:15, ...	Pass
T14	Unsupported slot length (45 min)	Rejected (400)	Pass
T15	Reefer load requests availability	Only the reefer dock is listed	Pass
T16	Dry-van truck books the reefer-only dock	Rejected (400)	Pass
T17	Booking during a maintenance window	Slot marked MAINTENANCE; booking rejected	Pass
T18	Booking on a blackout date	Day closed with reason; booking rejected	Pass
Lifecycle and workflows			
T19	COMPLETED → PENDING; CANCELLED → APPROVED	Both rejected (400)	Pass
T20	Carrier cancels a truck already at the dock	Rejected (400)	Pass
T21	Mark no-show before the slot starts	Rejected (400)	Pass
T22	Weekly series of 3 weeks	3 appointments created	Pass
T23	Join waitlist for a slot that still has space	Rejected (409)	Pass
T24	Cancel a booking with a carrier waiting	Waitlisted carrier auto-booked; entry FULFILLED; notified	Pass
T25	Reschedule to a free slot / to a full slot	200 with history / 409	Pass
T26	Gate check-in, dock start, complete	Timestamps stored; arrival ON_TIME	Pass
T27	Reliability after one on-time trip	Score 100, tier A	Pass
T28	Cancel remaining weekly series	2 remaining appointments cancelled	Pass

ID	Test case	Expected result	Status
T29	Audit log after the workflow	All action types recorded	Pass
T30	Analytics for the period	KPIs, per-dock utilisation and 7×24 heatmap returned	Pass

7.3 Concurrency Experiment

To verify that test T07 is meaningful, the same test was run with both protection layers temporarily disabled. In that case the reservation logic behaves like a naive "check then insert" inside a transaction.

Table 7.2: Double-booking experiment: 40 simultaneous requests across two servers

Configuration	Capacity-1 slot	Capacity-3 slot
Naive check-then-insert (protections disabled)	15 (**14 over**)	13 (**10 over**)
SmartDock reservation algorithm	**1**	**3**

7.4 Performance

Response times were measured from a client in India against the live deployment (Render free tier, Oregon region; MongoDB Atlas M0, Mumbai region), 15 sequential requests per endpoint after a warm-up request. The health check, which does no database work, represents the network round-trip time.

Table 7.3: Live API response times

Endpoint	Median	90th percentile
Health check	666 ms	757 ms
Slot availability (1 day)	1,338 ms	1,433 ms
Yard schedule (1 day)	1,575 ms	1,904 ms
Analytics (30 days)	4,300 ms	6,719 ms

During testing the analytics endpoint initially took about 5.8 s (median). Profiling showed that a new time-zone formatter object was created for every slot conversion; caching these objects reduced the median to the value above. The remaining latency is dominated by geography rather than computation: the free web service runs in Render's Oregon (USA) region while the database is in Mumbai, so every database query crosses the Pacific (roughly 0.2–0.3 s per round trip), and the health check alone shows a 0.5–0.7 s round trip from India

to the server. The analytics request also transfers about 900 appointment documents. Moving the web service to an Asian region (e.g. Render Singapore) next to the database, caching analytics results and using a paid instance are the identified improvements; even on the free tier, booking and yard operations complete in under two seconds.

7.5 Validation Against Objectives

Table 7.4: Objectives and how they were validated

Objective	**Evidence**
Multi-tenancy and access control	T01–T06
Configurable scheduling, holidays, maintenance	T11–T14, T17, T18
Load–dock matching	T15, T16
Capacity guarantee under concurrency	T07–T10, Table 7.2
Recurring, rescheduling, waitlist	T22–T25, T28
Yard execution, no-shows, reliability	T19–T21, T26, T27
KPIs, audit, notifications	T24, T29, T30; Chapters 6 and 8
Cloud deployment	Live smoke test; `https://smartdock.onrender.com`

CHAPTER 8

Results and Discussion

8.1 Operational KPIs from the Deployed System

Table 8.1 lists the KPIs reported by the live system's analytics module for the Chennai DC over the 30 days from 6 October to 4 November 2026. The appointments in this period come from the demo data generator, which models five carriers with different punctuality profiles, morning and afternoon demand peaks, and realistic gate-to-dock and handling times. The figures therefore demonstrate what the system measures and how; they are not measurements from a real warehouse.

Table 8.1: KPIs reported by SmartDock (Chennai DC, 30 days, demo data)

KPI	Value	KPI	Value
Appointments	910	Average per day	30.3
Completed	799	Pallets handled	15,151
Dock utilisation	40.0%	Offered truck-slots	2,184
On-time arrival rate	78.6%	No-show rate	8.6%
Cancellation rate	4.0%	Avg. turnaround (gate to gate)	57 min
Avg. yard wait (gate to door)	17 min	Avg. dock time (handling)	40 min

Discussion. A 40% utilisation with an 8.6% no-show rate shows that capacity is available but is partly reserved by trucks that never arrive — a direct, quantified case for the waitlist (which reallocates released capacity) and for using the reliability score in carrier negotiations. The demand heatmap (Figure 6.4) and busiest hours (10:00–11:00 and 15:00–16:00) show where labour should be concentrated and which hours carriers could be steered away from. The split of turnaround into a 17-minute yard wait and a 40-minute dock time tells the manager whether to work on gate and yard flow or on dock productivity.

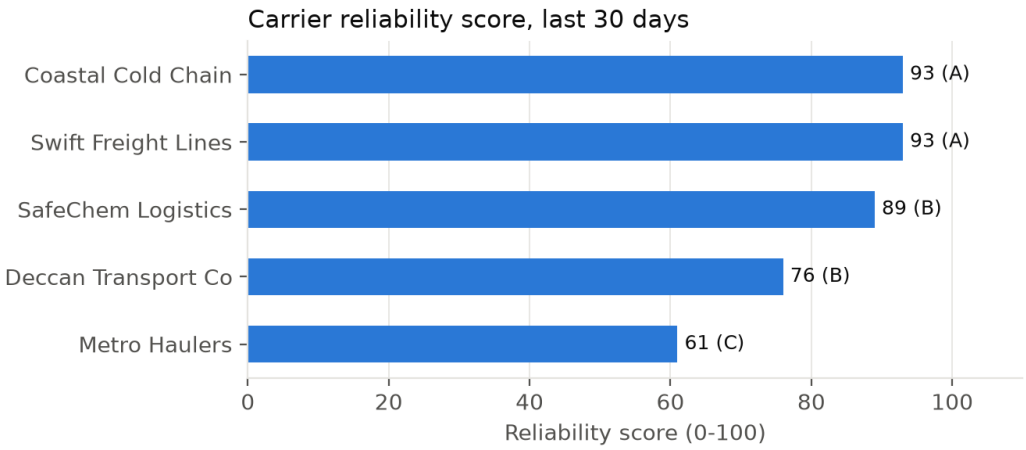


Figure 8.1: Carrier reliability scores computed by the system

Figure 8.1 shows how the scorecard separates carriers: Swift Freight Lines and Coastal Cold Chain score 93 (tier A) with on-time rates above 90%, while Metro Haulers scores 61 (tier C) with a 58% on-time rate and the most no-shows. These differences are exactly the input a procurement team needs for carrier selection and performance clauses.

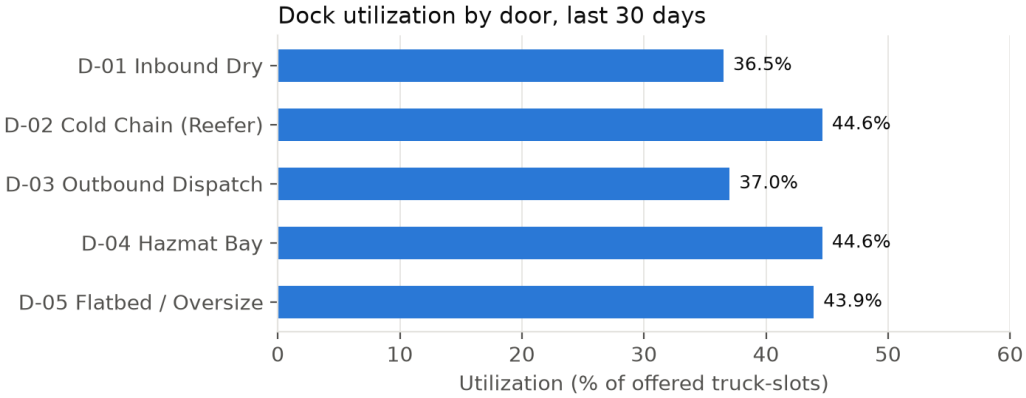


Figure 8.2: Utilisation by dock door

Single-capacity specialised doors (reefer, hazmat and flatbed, about 44–45%) are relatively busier than the two-truck general doors (about 37%), suggesting that specialised capacity, not total capacity, would become the constraint first as volume grows.

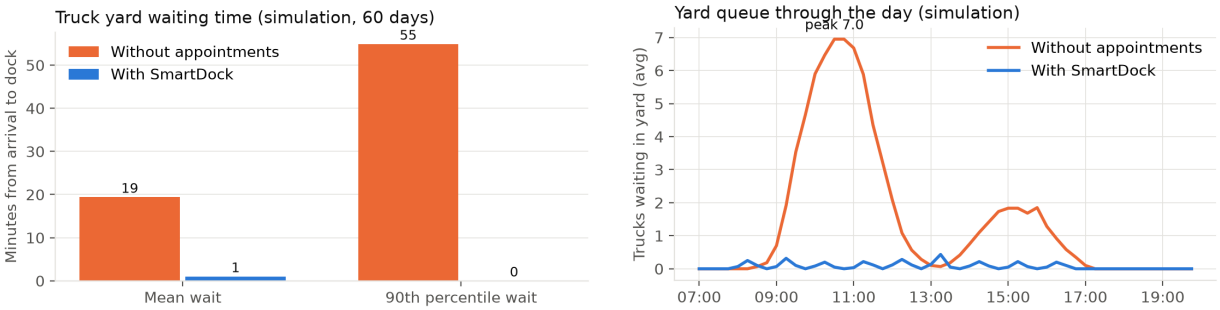
8.2 Before-versus-After Comparison (Simulation)

To estimate the effect of appointment scheduling on yard congestion, a discrete-event simulation [10] was run (script `docs/analysis/make_figures.py`). The model represents one DC day from 07:00 to 19:00 with six identical dock doors and 60 trucks; handling time is triangular (25, 40, 55 minutes) plus 5 minutes turnover; trucks are served first-come-first-served at the first free door. Sixty days were simulated for each scenario with the same random seed.

- **Before (no appointments):** carriers arrive at their preferred time, drawn from a morning peak around 09:30 (55%) and an afternoon peak around 14:30 (45%).
- **After (SmartDock):** each carrier books the free 60-minute slot nearest its preferred time, with at most six trucks per slot (one per door), and arrives around the slot start with the punctuality mix observed in the demo data (80% within −15 to +12 minutes, 15% late by 16–55 minutes, 5% no-show).

Table 8.2: Simulated yard performance, 60 days per scenario

Metric	Before	After	Change
Mean wait from arrival to dock	19.4 min	1.0 min	−95%
90th percentile wait	54.9 min	0.0 min	−100%
Maximum wait observed	116.3 min	37.7 min	−68%
Trucks waiting more than 30 min	27.7%	0.3%	−27.4 pts
Peak average yard queue	7.0 trucks	0.4 trucks	−94%



(a) Waiting time

(b) Yard queue through the day

Figure 8.3: Simulated effect of appointment scheduling

Discussion. With the same docks and the same work, smoothing arrivals removes almost all queueing: the morning and afternoon peaks that create a queue of about seven trucks disappear because capacity is allocated before trucks arrive. The residual waits are caused by late trucks arriving while the next appointment is already at the door, which is precisely what the late-arrival tracking and reliability score are designed to discourage. In money terms, each truck-hour of waiting avoided reduces detention charges, driver hours and idling emissions — the benefits reported for port appointment systems [6, 8].

Threats to validity. The simulation is deliberately simple: identical doors, one truck type, a fixed arrival pattern and perfect slot discipline for punctual trucks. In practice the gain depends on enforcement and carrier compliance [6], on handling-time variability and on how peaky demand really is. The numbers should be read as an indication of the order of magnitude, to be confirmed by a pilot at a real site.

8.3 Correctness Result

The most important technical result is the double-booking experiment (Section 7.3): under 40 simultaneous requests split across two servers, the naive approach admitted 15 trucks into a one-truck slot, whereas SmartDock admitted exactly one (and exactly three into a three-truck slot). For a warehouse this is the difference between a reliable schedule and a schedule that fails exactly at the busiest moments.

CHAPTER 9

Benefits, Conclusion and Future Work

9.1 Benefits and Real-World Applications

Table 9.1: Supply-chain impact of SmartDock

Dimension	Impact
Cost	Less truck waiting means lower detention charges and driver overtime; labour can be rostered to the known appointment load (heatmap, busiest hours) instead of covering unpredictable peaks; free and open-source software avoids licence fees.
Visibility	Both sides see the same schedule in real time: carriers see free capacity and status; managers see every truck's position (expected, in yard, at door, departed) and live KPIs.
Speed	Self-service booking replaces phone and e-mail; the simulation indicates that yard waiting falls from about 19 minutes to about 1 minute per truck; waitlist auto-assignment refills cancelled capacity without manual effort.
Traceability	Every appointment carries PO/ASN references, timestamps for each yard event and a complete audit trail, supporting detention disputes, compliance and root-cause analysis.
Reliability	Conflict-free scheduling (including dock compatibility and hazmat safety) and a carrier reliability score that creates accountability and supports carrier selection.
Sustainability	Shorter queues mean less engine idling and local emissions near the site [6].

Applications. Distribution centres of retailers and e-commerce companies (inbound supplier deliveries, outbound store or courier pickups); cold-chain warehouses (reefer-only doors and pharma 2–8 °C loads); chemical and fuel depots (hazmat bays); third-party logistics (3PL) providers running several client warehouses on one platform (multi-tenancy); manufacturing plants receiving just-in-time inbound parts; and, with minor changes, container freight stations and inland container depots.

9.2 Conclusion

This project designed, implemented, tested and deployed **SmartDock**, a multi-tenant dock appointment scheduling and yard management system. Mapping back to the objectives:

- Multi-tenancy with per-warehouse carrier approval and strict tenant isolation was implemented and verified by automated tests.

- A configurable scheduling engine supports slot lengths of 15–90 minutes, turnover buffers, holidays and maintenance windows across time zones.
- Loads are matched to doors by direction, trailer type and hazmat certification.
- Slot capacity is guaranteed under concurrency across multiple servers; a controlled experiment confirmed the guarantee and demonstrated the failure of the naive approach.
- Recurring series, rescheduling and an automatic FIFO waitlist cover realistic carrier needs.
- Yard events are digitised, late arrivals and no-shows are detected automatically, and carriers receive a reliability score.
- Managers obtain KPIs, heatmaps, scorecards, CSV/PDF reports and a gate sheet; every action is audited and notified.
- The system is live on the cloud and passes 58 end-to-end checks.

SmartDock shows that a focused information system at the warehouse–transport interface can convert an uncontrolled, phone-based process into a measured, collaborative one, improving cost, visibility, speed and traceability in the supply chain.

9.3 Project Links

Live system	`https://smartdock.onrender.com`
Source code (GitHub)	`https://github.com/RamaguruS/Dock`
Demo logins	`admin@smartdock.demo`, `carrier@smartdock.demo` (password `Demo@123`)
Run locally	`npm run dev:demo` (server) after building the client
Run tests	`npm run verify:critical` (58 end-to-end checks)

9.4 Future Enhancements

- **AI/ML:** predict no-show probability and arrival delay per carrier, lane and time of day; recommend slots and automatically overbook slightly where no-shows are likely; forecast daily inbound volume for labour planning.
- **Optimisation:** automatic assignment of trucks to doors (minimising yard wait and travel inside the building) and load-dependent slot lengths.
- **IoT and computer vision:** automatic gate check-in by number-plate recognition, RFID or GPS geofencing; dock-door sensors for real dock start and end times; reefer temperature monitoring.
- **Integration:** import purchase orders and ASNs from ERP/WMS, push appointments to TMS, and provide webhooks and a public API for carriers.

- **Mobile app:** a driver app with turn-by-turn directions to the assigned door, digital proof of delivery and live status; installable as a progressive web app.
- **Commercial features:** detention and no-show fee calculation, carrier contracts and service-level agreements, multi-language support.
- **Scalability and operations:** real-time updates with WebSockets instead of polling, caching of analytics, a super-administrator console for 3PL operators, horizontal scaling on a paid plan, and automated CI tests on every commit.

Bibliography

[1] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning, and Operation*, 6th ed. Pearson, 2016.

[2] M. Christopher, *Logistics & Supply Chain Management*, 5th ed. Pearson, 2016.

[3] D. Ivanov, A. Tsipoulanidis and J. Schönberger, *Global Supply Chain and Operations Management: A Decision-Oriented Introduction to the Creation of Value*, 2nd ed. Springer, 2019.

[4] J. Gu, M. Goetschalckx and L. F. McGinnis, "Research on warehouse operation: A comprehensive review," *European Journal of Operational Research*, vol. 177, no. 1, pp. 1–21, 2007.

[5] N. Boysen and M. Fliedner, "Cross dock scheduling: Classification, literature review and research agenda," *Omega*, vol. 38, no. 6, pp. 413–422, 2010.

[6] G. Giuliano and T. O'Brien, "Reducing port-related truck emissions: The terminal gate appointment system at the Ports of Los Angeles and Long Beach," *Transportation Research Part D: Transport and Environment*, vol. 12, no. 7, pp. 460–473, 2007.

[7] N. Huynh and C. M. Walton, "Robust scheduling of truck arrivals at marine container terminals," *Journal of Transportation Engineering*, vol. 134, no. 8, pp. 347–353, 2008.

[8] E. Zehendner and D. Feillet, "Benefits of a truck appointment system on the service quality of inland transport modes at a multimodal container terminal," *European Journal of Operational Research*, vol. 235, no. 2, pp. 461–469, 2014.

[9] M.-H. Phan and K. H. Kim, "Negotiating truck arrival times among trucking companies and a container terminal," *Transportation Research Part E: Logistics and Transportation Review*, vol. 75, pp. 132–144, 2015.

[10] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. McGraw-Hill, 2015.

[11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017, ch. 7 (Transactions: write skew and phantoms).

[12] MongoDB, Inc., "Transactions," *MongoDB Manual*. [Online]. Available: `https://www.mongodb.com/docs/manual/core/transactions/`

[13] OWASP Foundation, "OWASP Top 10:2021 – A01 Broken Access Control." [Online]. Available: `https://owasp.org/Top10/`

[14] Meta Open Source, "React documentation." [Online]. Available: `https://react.dev`

[15] OpenJS Foundation, “Express – Node.js web application framework.” [Online]. Available: `https://expressjs.com`

[16] Render, “Blueprint specification (render.yaml).” [Online]. Available: `https://render.com/docs/blueprint-spec`

CHAPTER A

Important Code Snippets

The listings below are taken directly from the project source code (`https://github.com/RamaguruS/Dock`). Line numbers refer to the original files.

Listing A.1: Slot grid for one local date: slot length, buffer, holidays (`server/src/services/scheduling.ts`)

```
124 export function generateSlotGrid(warehouse: WarehouseSchedule, date: string): SlotGrid {
125   const blackout = (warehouse.blackoutDates ?? []).find((b) => b.date === date);
126   if (blackout) {
127     return { closed: true, message: `Closed: ${blackout.reason}`, slots: [] };
128   }
129
130   const [year, month, day] = date.split('-').map(Number);
131   const dayOfWeek = DAY_NAMES[new Date(Date.UTC(year, month - 1, day)).getUTCDay()];
132   const opHours = warehouse.operatingHours.find((oh) => oh.day === dayOfWeek);
133   if (!opHours || !opHours.isOpen) {
134     return { closed: true, message: 'Warehouse is closed on this day', slots: [] };
135   }
136
137   const duration = warehouse.slotDurationMinutes || 60;
138   const buffer = warehouse.bufferMinutes || 0;
139   const open = toMinutes(opHours.start);
140   const close = toMinutes(opHours.end);
141
142   const slots: GridSlot[] = [];
143   for (let t = open; t + duration <= close; t += duration + buffer) {
144     const startLocal = toHHMM(t);
145     const endLocal = toHHMM(t + duration);
146     slots.push({
147       startLocal,
148       endLocal,
149       startUTC: localTimeToUTC(date, startLocal, warehouse.timezone),
150       endUTC: localTimeToUTC(date, endLocal, warehouse.timezone),
151     });
152   }
153   return { closed: false, slots };
154 }
```

Listing A.2: Daylight-saving-safe conversion of local slot times to UTC (`scheduling.ts`)

```
33 export function localTimeToUTC(dateStr: string, timeStr: string, timezone: string): Date {
34   const formatter = cachedFormatter('en-US', {
35     timeZone: timezone,
36     year: 'numeric',
37     month: 'numeric',
38     day: 'numeric',
39     hour: 'numeric',
40     minute: 'numeric',
41     second: 'numeric',
42     hour12: false,
43   });
44
45   const [year, month, day] = dateStr.split('-').map(Number);
46   const [hour, minute] = timeStr.split(':').map(Number);
```

```
  const target = Date.UTC(year, month - 1, day, hour, minute);

  let utcDate = new Date(target);
  for (let i = 0; i < 3; i++) {
    const parts: Record<string, string> = {};
    formatter.formatToParts(utcDate).forEach((p) => { parts[p.type] = p.value; });
    let fHour = parseInt(parts.hour);
    if (fHour === 24) fHour = 0;
    const seen = Date.UTC(parseInt(parts.year), parseInt(parts.month) - 1, parseInt(parts.day), fHour,
     parseInt(parts.minute));
    const diffMs = target - seen;
    if (diffMs === 0) break;
    utcDate = new Date(utcDate.getTime() + diffMs);
  }
  return utcDate;
}
```

Listing A.3: Load–dock compatibility: direction, trailer type, hazmat (`server/src/models/Dock.ts`)

```
export interface LoadRequirements {
  direction?: string;
  trailerType?: string;
  hazmat?: boolean;
}

/** Can this dock serve a load with these requirements? */
export const isDockCompatible = (
  dock: Pick<IDock, 'direction' | 'supportedTrailerTypes' | 'hazmatCertified'>,
  load: LoadRequirements
): boolean => {
  if (load.direction && dock.direction !== 'BOTH' && dock.direction !== load.direction) return false;
  if (load.trailerType && !(dock.supportedTrailerTypes ?? []).includes(load.trailerType as TrailerType))
    return false;
  if (load.hazmat && !dock.hazmatCertified) return false;
  return true;
};
```

Listing A.4: Atomic capacity reservation: in-process dock lock and transactional dock write lock (`scheduling.ts`)

```
export class SlotFullError extends Error {}

const dockLocks = new Map<string, Promise<void>>();

async function withDockLock<T>(dockId: string, fn: () => Promise<T>): Promise<T> {
  const previous = dockLocks.get(dockId) ?? Promise.resolve();
  let release!: () => void;
  const current = new Promise<void>((resolve) => { release = resolve; });
  const tail = previous.then(() => current);
  dockLocks.set(dockId, tail);

  await previous;
  try {
    return await fn();
  } finally {
    release();
    if (dockLocks.get(dockId) === tail) dockLocks.delete(dockId);
  }
}

```

```
366 let transactionsSupported: boolean | undefined;
367
368 const isTransactionUnsupportedError = (error: any): boolean =>
369   error?.message?.includes('replica set') ||
370   error?.message?.includes('Transaction numbers') ||
371   error?.code === 20 ||
372   error?.codeName === 'IllegalOperation';
373
374 /**
375  * Atomically: check that fewer than `capacity` active bookings overlap [start, end) on the dock
376  * (ignoring `excludeBookingId`, used when rescheduling), then run `write`.
377  * Throws SlotFullError when the slot is full.
378  */
379 export async function reserveCapacity<T>(
380   params: { dockId: string; startTime: Date; endTime: Date; capacity: number; excludeBookingId?: string },
381   write: (session?: mongoose.ClientSession) => Promise<T>
382 ): Promise<T> {
383   const { dockId, startTime, endTime, capacity, excludeBookingId } = params;
384
385   const checkAndWrite = async (session?: mongoose.ClientSession) => {
386     if (session) {
387       // Take a write lock on the dock document (see comment above)
388       await Dock.updateOne({ _id: dockId }, { $inc: { bookingLockVersion: 1 } }, { session });
389     }
390     const filter: Record<string, unknown> = {
391       dockId,
392       status: { $nin: INACTIVE_STATUSES },
393       startTime: { $lt: endTime },
394       endTime: { $gt: startTime },
395     };
396     if (excludeBookingId) filter._id = { $ne: excludeBookingId };
397
398     const overlapping = await Booking.countDocuments(filter).session(session ?? null);
399     if (overlapping >= capacity) throw new SlotFullError('Selected dock slot is fully booked');
400     return write(session);
401   };
402
403   return withDockLock(dockId, async () => {
404     if (transactionsSupported !== false) {
405       const session = await mongoose.startSession();
406       try {
407         let result: T | undefined;
408         await session.withTransaction(async () => {
409           result = await checkAndWrite(session);
410         });
411         transactionsSupported = true;
412         return result as T;
413       } catch (error) {
414         if (!isTransactionUnsupportedError(error)) throw error;
415         transactionsSupported = false;
416         logger.warn(
417           '[BOOKING] MongoDB transactions are unavailable (standalone server). Falling back to the
      in-process ' +
418           'dock lock, which is only safe with a single server instance. Use a replica set (e.g. Atlas) in
      production.'
419         );
420       } finally {
421         await session.endSession();
422       }
423     }
424     return checkAndWrite();
425   });
```

```
426 }
```

Listing A.5: Appointment lifecycle: allowed status transitions (`server/src/controllers/bookingController.ts`)

```
58 /**
59  * Appointment lifecycle (yard flow):
60  *   PENDING → APPROVED → CHECKED_IN (gate) → IN_PROGRESS (at door) → COMPLETED
61  * COMPLETED / CANCELLED / REJECTED / NO_SHOW are terminal, so a released slot can never be
62  * re-occupied by reviving an old booking (that would bypass the capacity check).
63  */
64 const ALLOWED_TRANSITIONS: Record<BookingStatus, BookingStatus[]> = {
65   PENDING:     ['APPROVED', 'REJECTED', 'CANCELLED'],
66   APPROVED:    ['CHECKED_IN', 'IN_PROGRESS', 'CANCELLED', 'NO_SHOW'],
67   CHECKED_IN:  ['IN_PROGRESS', 'CANCELLED'],
68   IN_PROGRESS: ['COMPLETED'],
69   COMPLETED:   [],
70   CANCELLED:   [],
71   REJECTED:    [],
72   NO_SHOW:     [],
73 };
```

Listing A.6: Gate check-in: on-time or late arrival (`bookingController.ts`)

```
362     if (status === 'CHECKED_IN' || (status === 'IN_PROGRESS' && !booking.checkedInAt)) {
363       const lateBy = Math.round((now.getTime() - booking.startTime.getTime()) / 60000);
364       set.checkedInAt = now;
365       set.lateByMinutes = Math.max(0, lateBy);
366       set.arrivalStatus = lateBy > (warehouse.lateThresholdMinutes ?? 15) ? 'LATE' : 'ON_TIME';
367     }
368     if (status === 'IN_PROGRESS') set.dockStartedAt = now;
369     if (status === 'COMPLETED') set.completedAt = now;
```

Listing A.7: Waitlist: first-in-first-out assignment of released capacity (`server/src/services/waitlist.ts`)

```
40     for (const entry of entries) {
41       if (!isDockCompatible(dock, entry)) continue;
42       if (!(await hasApprovedAccess(entry.vendorId.toString(), warehouseId))) continue;
43
44       try {
45         const booking = await reserveCapacity(
46           { dockId, startTime, endTime, capacity: dock.maxCapacityPerHour },
47           async (session) => {
48             const [created] = await Booking.create(
49               [
50                 {
51                   warehouseId,
52                   dockId,
53                   vendorId: entry.vendorId,
54                   truckNumber: entry.truckNumber,
55                   driverName: entry.driverName,
56                   driverPhone: entry.driverPhone,
57                   startTime,
58                   endTime,
59                   direction: entry.direction,
60                   trailerType: entry.trailerType,
61                   hazmat: entry.hazmat,
62                   referenceNumber: entry.referenceNumber,
63                   palletCount: entry.palletCount,
64                   status: 'PENDING',
```

```
            },
          ],
          { session }
        );
        // Claim the entry inside the same transaction so it can't be fulfilled twice
        const claimed = await WaitlistEntry.updateOne(
          { _id: entry._id, status: 'WAITING' },
          { $set: { status: 'FULFILLED', fulfilledBookingId: created._id } },
          { session }
        );
        if (claimed.modifiedCount === 0) throw new Error('WAITLIST_ALREADY_CLAIMED');
        return created;
      }
    );

    assigned++;
    const when = formatSlot(startTime, warehouse.timezone);
    await notifyUser(entry.vendorId, {
      type: 'WAITLIST_ASSIGNED',
      title: 'Waitlist slot assigned',
      message: `A slot opened up at ${warehouse.name} (${dock.dockName}, ${when}). Your truck ${entry.truckNumber} has been booked and is awaiting warehouse approval.`,
      bookingId: booking._id,
    });
    await notifyWarehouseAdmins(warehouseId, {
      type: 'BOOKING_CREATED',
      title: 'Waitlist auto-assignment',
      message: `Truck ${entry.truckNumber} was auto-assigned from the waitlist to ${dock.dockName} at ${when}. Approval needed.`,
      bookingId: booking._id,
    });
    await audit({
      system: true,
      warehouseId,
      action: 'WAITLIST_AUTO_ASSIGNED',
      entityType: 'BOOKING',
      entityId: booking._id,
      message: `Waitlist entry for truck ${entry.truckNumber} auto-assigned to ${dock.dockName} at ${when}`,
      metadata: { waitlistEntryId: entry._id.toString() },
    });
  } catch (error) {
    if (error instanceof SlotFullError) break; // no more capacity
    if ((error as Error).message === 'WAITLIST_ALREADY_CLAIMED') continue;
    throw error;
  }
}
```

Listing A.8: Carrier reliability score (`server/src/services/reliability.ts`)

```
const result = new Map<string, CarrierScore>();
for (const id of vendorIds) {
  const r = rows.find((row) => row._id.toString() === id.toString());
  const onTime = r?.onTime ?? 0;
  const late = r?.late ?? 0;
  const noShows = r?.noShows ?? 0;
  const due = onTime + late + noShows;
  const score = due === 0 ? null : Math.round(((onTime + 0.5 * late) / due) * 100);
  result.set(id.toString(), {
    vendorId: id.toString(),
    totalAppointments: r?.total ?? 0,
    completed: r?.completed ?? 0,
```

```
      onTime,
      late,
      noShows,
      cancelled: r?.cancelled ?? 0,
      lateCancellations: r?.lateCancellations ?? 0,
      avgLateMinutes: late ? Math.round((r.lateMinutesSum ?? 0) / late) : null,
      onTimeRate: pct(onTime, onTime + late),
      noShowRate: pct(noShows, due),
      reliabilityScore: score,
      ...tierFor(score),
    });
  }
  return result;
}
```

Listing A.9: Background job: automatic no-show detection (`server/src/services/jobs.ts`)

```
async function markNoShows(now: Date) {
  const warehouses = await Warehouse.find().select('name timezone noShowGraceMinutes');
  for (const w of warehouses) {
    const cutoff = new Date(now.getTime() - (w.noShowGraceMinutes ?? 60) * 60000);
    const due = await Booking.find({ warehouseId: w._id, status: 'APPROVED', startTime: { $lt: cutoff }
    }).limit(200);
    for (const b of due) {
      const updated = await Booking.findOneAndUpdate(
        { _id: b._id, status: 'APPROVED' },
        { $set: { status: 'NO_SHOW' } },
        { new: true }
      );
      if (!updated) continue;
      const when = formatSlot(b.startTime, w.timezone);
      await audit({
        system: true,
        warehouseId: w._id,
        action: 'BOOKING_NO_SHOW',
        entityType: 'BOOKING',
        entityId: b._id,
        message: `Truck ${b.truckNumber} did not arrive for ${when}; marked NO_SHOW after
    ${w.noShowGraceMinutes} min grace`,
      });
      await notifyUser(b.vendorId, {
        type: 'BOOKING_NO_SHOW',
        title: 'Missed appointment (no-show)',
        message: `Truck ${b.truckNumber} did not check in for ${w.name}, ${when}. This affects your
    reliability score.`,
        bookingId: b._id,
      });
    }
  }
}
```

Listing A.10: Per-request tenant and warehouse-access control (`server/src/middleware/authMiddleware.ts`)

```
export const requireWarehouseAccess = (source: 'body' | 'query') => {
  return async (req: Request, res: Response, next: NextFunction): Promise<void> => {
    if (!req.user) {
      res.status(401).json({ error: 'Unauthorized' });
      return;
    }

```

```
78      const warehouseId = source === 'body' ? req.body?.warehouseId : req.query.warehouseId;
79      if (typeof warehouseId !== 'string' || !Types.ObjectId.isValid(warehouseId)) {
80        res.status(400).json({ error: 'A valid warehouseId is required' });
81        return;
82      }
83
84      const { role } = req.user;
85
86      if (role === 'SUPER_ADMIN') {
87        next();
88        return;
89      }
90
91      if (role === 'WAREHOUSE_ADMIN') {
92        if (req.user.warehouseId !== warehouseId) {
93          res.status(403).json({ error: 'Forbidden: You do not manage this warehouse' });
94          return;
95        }
96        next();
97        return;
98      }
99
100     try {
101       const user = await User.findById(req.user.userId).select('status');
102       if (!user || user.status !== 'APPROVED') {
103         res.status(403).json({ error: 'Forbidden: Your account is not active' });
104         return;
105       }
106
107       if (!(await hasApprovedAccess(req.user.userId, warehouseId))) {
108         res.status(403).json({ error: 'Forbidden: You are not approved to book at this warehouse' });
109         return;
110       }
111     } catch (error) {
112       res.status(500).json({ error: 'Internal Server Error during access check' });
113       return;
114     }
115
116     next();
117   };
118 };
```